

ChaatCheck

Indian Street Food Recognizer

T2.4 — Street Food Tier 1

"Point your phone at a thali — the app names every dish, estimates calories, and flags allergens in real time."

Course: Statistical Methods in AI (SMAI)
Assignment: Assignment 3 — Transfer Learning & Image Classification
Task: T2 — Indian Food Recognizer (Street Food, Tier 1)
Model: MobileNetV2 (Fine-tuned, 12 classes)
Framework: PyTorch + Streamlit
Dataset: Indian Food Images (Kaggle, 80 classes → 12 selected)

Team Members:

Karthik Nallala	2025201095
Suresh Kumar Vuyala	2025201051
Chaitanya Kadupukutla	2025201077

Abstract

This report presents **ChaatCheck**, an end-to-end deep learning application for recognising 12 categories of Indian street food from photographs. We fine-tune a MobileNetV2 backbone pre-trained on ImageNet using a two-stage transfer learning strategy: first training only the classifier head on frozen convolutional features, then partially unfreezing the last three convolutional blocks for domain adaptation. The model is trained on 1,260 images (50 original per class, augmented $3\times$ with random flips, rotations, and colour jitter) and evaluated on a held-out test set of 96 images, achieving a test accuracy of **83.33%** with a macro F1-score of **0.81**. Per-class analysis reveals that visually distinctive dishes (e.g., jalebi, $F1 = 1.00$) are classified near-perfectly, while visually similar fried items (e.g., daal_puri, $F1 = 0.33$) remain challenging. The trained model is deployed via a Streamlit web application that provides real-time predictions alongside estimated calorie counts, dietary labels, ingredient lists, and colour-coded allergen warnings.

Contents

1	Introduction & Problem Statement	4
1.1	Motivation	4
1.2	Scope — Task T2.4 (Street Food Tier 1)	4
2	Dataset Description	4
2.1	Source Dataset	4
2.2	Class Selection — 12 Street Food Classes	4
2.3	Dataset Split	5
3	Data Preparation Pipeline	6
3.1	Phase 1A — Acquisition & Splitting (<code>Phase_1_Data_Prep.ipynb</code>)	6
3.2	Phase 1B — Offline Augmentation (<code>Phase_1_Perform_Augmentation.ipynb</code>)	7
3.2.1	Augmentation Pipeline	7
3.2.2	Augmented Dataset Counts	8
4	Model Architecture	8
4.1	Base Model — MobileNetV2	8
4.1.1	Depthwise Separable Convolutions	8
4.1.2	Inverted Residual Blocks with Linear Bottlenecks	8
4.2	Pipeline Architecture Diagram	9
4.3	Transfer Learning Strategy	10
4.3.1	Stage 1 — Classifier-Only Training	10
4.3.2	Stage 2 — Partial Unfreezing (Gradual Fine-Tuning)	11
4.4	Inference Transform	11
5	Training	11
5.1	Training Configuration	12
5.2	Hyperparameter Rationale	12
5.3	Training & Validation Results	12
5.4	Learning Curve	13
5.5	Observations	14
6	Inference System (<code>model_inference.py</code>)	15
7	Streamlit Application (<code>app.py</code>)	15
7.1	Architecture	15
7.2	UI Layout	15
7.3	Allergen Badge System	16
7.4	Streamlit App Screenshots	16
7.5	Metadata Store	17
8	Repository Structure	17
9	Results Summary	18
9.1	Per-Class Classification Report	18
9.1.1	Key Observations	19
9.2	Confusion Matrix	19
9.3	Error Analysis	20

9.3.1	Failure Mode 1: The Fried-Dough Cluster	20
9.3.2	Failure Mode 2: High Variance in Plating (litti_chokha)	20
9.3.3	Failure Mode 3: Well-Separated Classes	20
10	Limitations & Future Work	20
10.1	Current Limitations	20
10.2	Proposed Improvements	21
11	Conclusion	22
	References	22

1. Introduction & Problem Statement

ChaatCheck is an end-to-end deep learning system for classifying Indian street food images. Given a photograph of food, the system:

- Predicts the **dish name** from 12 possible street food classes
- Returns a **confidence score** for the prediction
- Displays **calorie estimates** per serving
- Shows **colour-coded allergen badges** (Gluten, Dairy, Nuts)
- Lists **key ingredients**

1.1 Motivation

India's street food ecosystem is enormous and culturally diverse. Identifying dishes by photograph is non-trivial due to variations in preparation, plating, lighting, and regional style. Building an automated classifier has practical applications in dietary tracking, menu digitisation, and allergen safety.

1.2 Scope — Task T2.4 (Street Food Tier 1)

This project implements Tier 1 of Task T2 from the SMAI assignment specification, targeting 12 street food classes drawn from the master 80-class Indian Food Images dataset. The 12 classes were selected for their distinct visual signatures and prevalence in North and South Indian street food culture.

Dataset Source

Indian Food Images Dataset — Kaggle (iamsouravbanerjee). 4,000+ images across 80 food categories. Licensed for educational use.

2. Dataset Description

2.1 Source Dataset

The primary dataset contains 4,000+ images spanning 80 Indian food classes. Images vary in resolution, aspect ratio, background clutter, and lighting conditions — making the task representative of real-world deployment challenges.

2.2 Class Selection — 12 Street Food Classes

From the 80 available classes, 12 were selected based on street food prevalence and visual distinctiveness. Table 1 lists all 12 classes with their nutritional metadata.

Table 1: Selected Street Food Classes with Metadata

#	Class Name	Calories (kcal)	Diet	Allergens	Key Ingredients
1	aloo_tikki	180–220	Veg	Gluten	Potato, bread crumbs, spices
2	bhatura	300–350	Veg	Gluten	Maida, yogurt, oil
3	chana_masala	220–260	Vegan	None	Chickpeas, tomato, spices
4	daal_puri	260–300	Veg	Gluten	Wheat flour, lentils, oil
5	jalebi	140–180	Veg	Gluten	Maida, sugar syrup, ghee
6	kachori	250–300	Veg	Gluten	Flour, lentils, oil
7	lassi	150–200	Veg	Dairy	Curd, sugar, cardamom
8	litti_chokha	300–350	Veg	Gluten	Wheat flour, sattu, ghee
9	poha	150–200	Vegan	Nuts	Flattened rice, peanuts, spices
10	rabri	300–400	Veg	Dairy, Nuts	Milk, sugar, dry fruits
11	kuzhi_paniyaram	200–250	Veg	None	Rice batter, lentils, oil
12	unni_appam	220–260	Veg	Gluten	Rice flour, banana, jaggery

2.3 Dataset Split

Images were split into train, validation, and test partitions using a **70/15/15** ratio, applied per class after random shuffling to ensure balanced representation. Table 2 shows per-class counts before augmentation.

Table 2: Dataset Split (Before Augmentation)

Class	Train	Val	Test	Total
aloo_tikki	35	7	8	50
bhatura	35	7	8	50
chana_masala	35	7	8	50
daal_puri	35	7	8	50
jalebi	35	7	8	50
kachori	35	7	8	50
lassi	35	7	8	50
litti_chokha	35	7	8	50
poha	35	7	8	50
rabri	35	7	8	50
kuzhi_paniyaram	35	7	8	50
unni_appam	35	7	8	50
Total	420	84	96	600

3. Data Preparation Pipeline

The data preparation phase was executed in two sequential Jupyter notebooks on Google Colab (GPU: NVIDIA T4).

3.1 Phase 1A — Acquisition & Splitting (Phase_1_Data_Prep.ipynb)

1. **Mount Google Drive** — all outputs persist across Colab sessions via `/content/drive/MyDrive/SMAI`
2. **Configure Kaggle API** — credentials set via environment variables and `~/.kaggle/kaggle.json`.
3. **Download dataset** — the full 355 MB archive is pulled directly using `kaggle datasets download`.
4. **Extract archive** — unzipped to `full_dataset_80_classes/`. The internal structure is `Indian Food Images/Indian Food Images/<class>/`.
5. **Filter to 12 classes** — only the 12 street food folders are copied; the remaining 68 classes are ignored.
6. **Split per class** — each class folder is shuffled and split at indices `int(N*0.70)` and `int(N*0.85)`.

```

1 for cls in street_food:
2     imgs = [f for f in os.listdir(src) if
3             f.lower().endswith(('.png', '.jpg', '.jpeg'))]
4     random.shuffle(imgs)
5     train_idx, val_idx = int(len(imgs)*0.7), int(len(imgs)*0.85)
6
7     for name, split in [
8         ('train', imgs[:train_idx]),

```

```
8         ('val', imgs[train_idx:val_idx]),
9         ('test', imgs[val_idx:]))
10     ]:
11         dst = os.path.join(output_dir, name, cls)
12         os.makedirs(dst, exist_ok=True)
13         for img in split:
14             shutil.copy(os.path.join(src, img),
15                           os.path.join(dst, img))
```

Listing 1: Core split logic from Phase_1_Data_Prep.ipynb

3.2 Phase 1B — Offline Augmentation (Phase_1_Perform_Augmentation.ipynb)

With only 35 images per class in training, the model is highly prone to overfitting. To mitigate this, offline augmentation was applied: each original image generates **2 additional augmented versions**, tripling the effective training set size.

3.2.1 Augmentation Pipeline

```
1 aug_pipeline = transforms.Compose([
2     transforms.RandomHorizontalFlip(p=0.5),
3     transforms.RandomRotation(degrees=15),
4     transforms.ColorJitter(brightness=0.2, contrast=0.2)
5 ])
```

Listing 2: Augmentation transforms applied per image

Augmented images are saved alongside originals with the naming convention `<basename>_aug1.jpg`, `<basename>_aug2.jpg`. Only training images are augmented; validation and test sets remain untouched to simulate real-world inference conditions.

3.2.2 Augmented Dataset Counts

Table 3: Dataset Counts After Augmentation

Class	Train (Orig)	Train (Aug)	Train Total	Val	Test	Grand Total
aloo_tikki	35	70	105	7	8	120
bhatura	35	70	105	7	8	120
chana_masala	35	70	105	7	8	120
daal_puri	35	70	105	7	8	120
jalebi	35	70	105	7	8	120
kachori	35	70	105	7	8	120
lassi	35	70	105	7	8	120
litti_chokha	35	70	105	7	8	120
poha	35	70	105	7	8	120
rabri	35	70	105	7	8	120
kuzhi_paniyaram	35	70	105	7	8	120
unni_appam	35	70	105	7	8	120
Total	420	840	1,260	84	96	1,440

4. Model Architecture

4.1 Base Model — MobileNetV2

MobileNetV2 [2] was selected over heavier alternatives (e.g., ResNet-50 at ≈ 25 M parameters) for its strong accuracy-to-parameter ratio (≈ 3.4 M parameters), making it well-suited for real-time inference in a lightweight Streamlit deployment. The architecture introduces two key innovations:

4.1.1 Depthwise Separable Convolutions

Standard convolutions apply a $k \times k$ filter across *all* input channels simultaneously, requiring $k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$ multiplications per spatial position. MobileNetV2 factorises this into two steps:

1. **Depthwise convolution** — a single $k \times k$ filter per input channel ($k^2 \cdot C_{\text{in}}$ operations).
2. **Pointwise convolution** — a 1×1 convolution to mix channels ($C_{\text{in}} \cdot C_{\text{out}}$ operations).

This reduces computation by a factor of approximately $\frac{1}{C_{\text{out}}} + \frac{1}{k^2}$, which for $k = 3$ and $C_{\text{out}} = 32$ yields a ~ 8 – $9\times$ speedup over standard convolutions.

4.1.2 Inverted Residual Blocks with Linear Bottlenecks

Unlike ResNet’s bottleneck (which *compresses* to a lower dimension, processes, then *expands*), MobileNetV2’s inverted residual blocks do the opposite: they **expand** the channel dimension via a 1×1 convolution (expansion factor $t = 6$), apply a depthwise 3×3

convolution in the expanded space, then **project back** to a narrow bottleneck. Residual connections are added between the narrow bottleneck layers.

Crucially, the final projection uses a **linear activation** (no ReLU) because ReLU destroys information in low-dimensional spaces — a key insight from the MobileNetV2 paper that preserves representational capacity in the bottleneck.

Table 4: MobileNetV2 Architecture Summary

Property	Value
Architecture	MobileNetV2
Pretrained weights	ImageNet (1,000 classes)
Input size	$224 \times 224 \times 3$
Last channel width	1,280
Modified layer	<code>classifier[1]</code>
Output classes	12 (street food)
Total parameters	$\approx 3.4\text{M}$

4.2 Pipeline Architecture Diagram

Figure 1 illustrates the end-to-end inference pipeline from raw input image to final prediction.

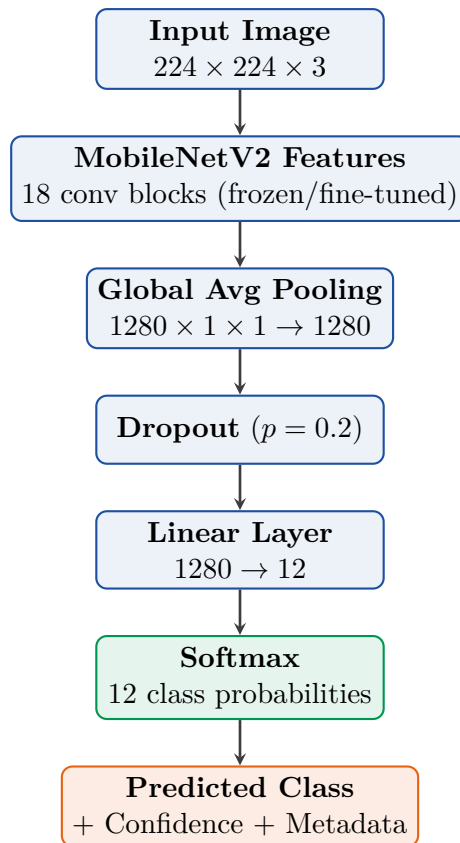


Figure 1: End-to-end inference pipeline: from input image to predicted dish with confidence score and metadata lookup

4.3 Transfer Learning Strategy

A **two-stage fine-tuning** approach was employed to balance feature reuse from ImageNet with domain adaptation to Indian street food images.

4.3.1 Stage 1 — Classifier-Only Training

All convolutional (feature extraction) layers were **frozen**, and only the newly added classifier head was trained. This allows the randomly initialised classifier to converge using the stable, high-quality features from ImageNet, without corrupting the pre-trained convolutional weights.

```

1 model = models.mobilenet_v2(pretrained=True)
2
3 # Freeze all convolutional layers
4 for param in model.parameters():
5     param.requires_grad = False
6
7 # Replace classifier head: 1280 -> 12 classes
8 model.classifier[1] = nn.Linear(model.last_channel, 12)
9
10 # Only classifier parameters are trainable
11 for param in model.classifier.parameters():
12     param.requires_grad = True
  
```

```
13
14 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Listing 3: Stage 1: Freeze backbone, train classifier only

4.3.2 Stage 2 — Partial Unfreezing (Gradual Fine-Tuning)

After the classifier converged (10 epochs), the **last 3 convolutional blocks** (`features[-3:]`) were unfrozen with a **reduced learning rate** (10^{-4}) to allow the deeper layers to adapt to food-specific features while avoiding catastrophic forgetting of the general-purpose ImageNet representations.

```
1 for param in model.features[-3:].parameters():
2     param.requires_grad = True
3
4 optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

Listing 4: Stage 2: Unfreeze last 3 blocks for fine-tuning

This two-stage strategy is a well-established transfer learning technique: the frozen-backbone stage prevents the randomly initialised classifier from sending noisy gradients through the pre-trained features, while the subsequent partial unfreezing allows domain-specific adaptation in the higher convolutional layers.

4.4 Inference Transform

At inference time, images are pre-processed with a deterministic pipeline (no augmentation):

```
1 transform = transforms.Compose([
2     transforms.Resize((224, 224)),
3     transforms.ToTensor(),
4     # Note: ImageNet normalisation can be added here for
5     # improved accuracy
6 ])
```

Listing 5: Inference pre-processing pipeline (`model_inference.py`)

Softmax is applied over the 12 logits to obtain class probabilities, and `torch.max` selects the predicted class and its confidence score.

5. Training

5.1 Training Configuration

Table 5: Training Hyperparameters

Hyperparameter	Value
Epochs	10
Batch size	≈ 32 (40 batches for 1,260 train images)
Optimiser	Adam
Loss function	Cross Entropy Loss
Hardware	Google Colab (NVIDIA T4 GPU)
Image size	224×224
Classes	12

5.2 Hyperparameter Rationale

- **Adam optimiser** was chosen over SGD because Adam’s per-parameter adaptive learning rates are well-suited for fine-tuning scenarios where the classifier head (randomly initialised) and the backbone (pre-trained) have very different gradient magnitudes. Adam converges faster on small datasets where SGD’s momentum-based updates may oscillate.
- **CrossEntropyLoss** combines **LogSoftmax** and **NLLoss** into a single numerically stable operation, and is the standard choice for multi-class classification with mutually exclusive labels.
- **Batch size 32** balances GPU memory utilisation (Google Colab T4 with 15 GB VRAM) with sufficient gradient noise to aid generalisation on a small dataset. Larger batches (64, 128) risk memorising the limited training data.
- **Learning rate (10^{-3} for Stage 1, 10^{-4} for Stage 2):** The $10\times$ reduction in Stage 2 ensures that the unfrozen convolutional layers are updated gently, preserving the useful ImageNet features while allowing domain-specific adaptation.

5.3 Training & Validation Results

Table 6 records the average training loss and validation accuracy at each epoch. The best model (Epoch 7, Val Acc = **83.33%**) was saved to `food_model.pth`.

Table 6: Per-Epoch Training Loss and Validation Accuracy

Epoch	Avg. Training Loss	Validation Accuracy	Notes
1	1.2369	78.57%	Rapid initial convergence
2	0.7392	79.76%	Loss drop of 40%
3	0.5421	82.14%	First significant val jump
4	0.4428	79.76%	Slight val regression
5	0.3686	82.14%	Recovery
6	0.3238	79.76%	Minor oscillation
7	0.2736	83.33%	Best model — saved
8	0.2447	80.95%	Slight drop after peak
9	0.2272	82.14%	Continued loss reduction
10	0.2121	82.14%	Training converged

5.4 Learning Curve

Figure 2 shows the training loss and validation accuracy over 10 epochs, rendered from the logged values.

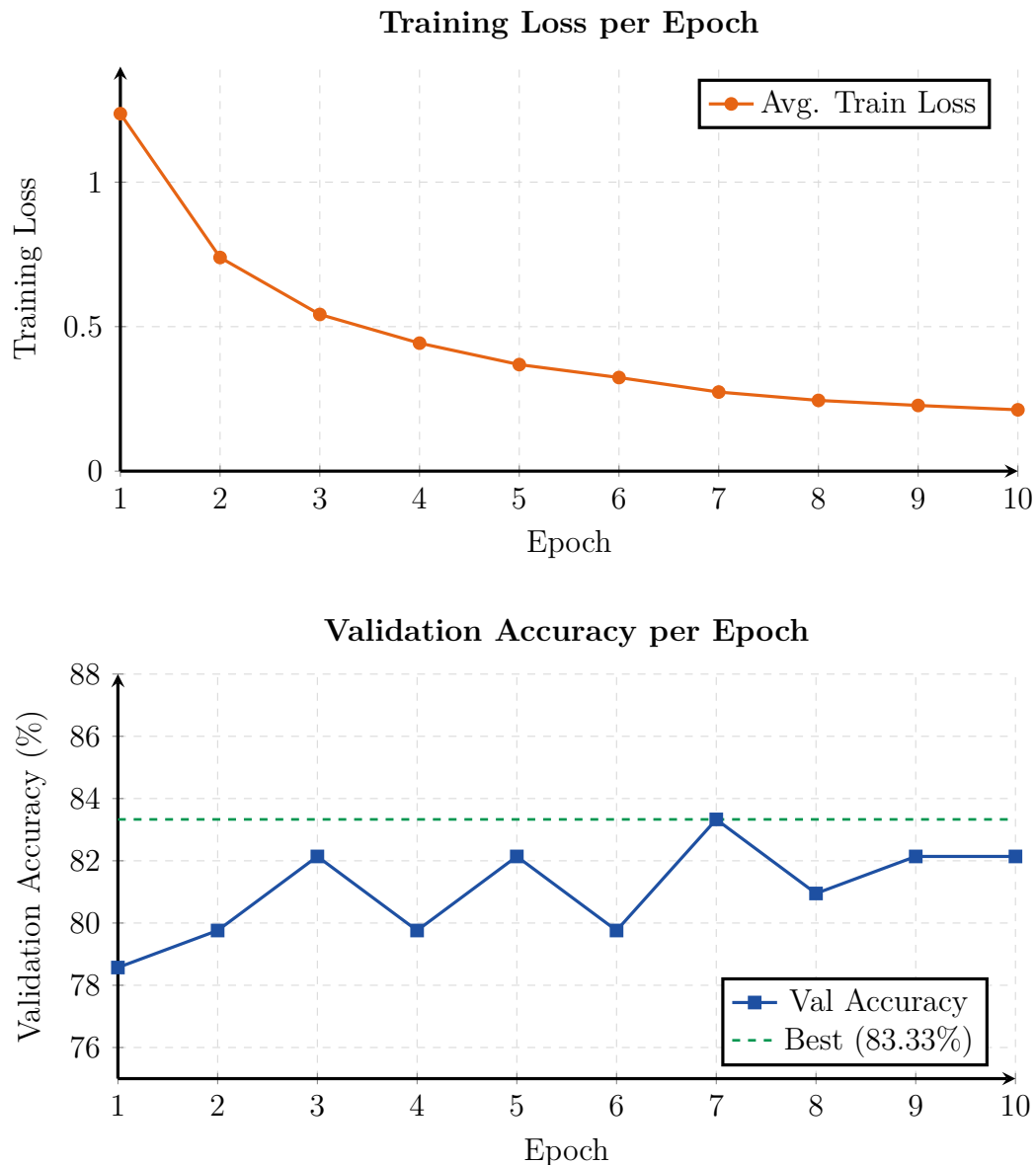


Figure 2: Training loss (top) and validation accuracy (bottom) across 10 epochs

5.5 Observations

- Training loss decreases **monotonically** from 1.24 to 0.21, indicating successful gradient-based optimisation.
- Validation accuracy **oscillates** between ≈ 79 –83%, suggesting moderate overfitting on the small dataset.
- The best validation accuracy of **83.33%** is achieved at Epoch 7, after which the model begins to slightly overfit.
- The gap between training convergence and validation plateau can be addressed by stronger regularisation or a larger dataset.

6. Inference System (model_inference.py)

The inference module is intentionally decoupled from the UI to ensure the model logic can be tested independently. It exposes two public functions:

- `load_model(path)` — Rebuilds the MobileNetV2 architecture with the 12-class head and loads the saved .pth weights. Sets model to `eval()` mode.
- `predict(image, model)` — Applies the inference transform, runs a single forward pass under `torch.no_grad()`, applies softmax, and returns `(class_name, confidence)`.

```

1 class_names = [
2     "aloo_tikki", "bhatura", "chana_masala", "daal_puri",
3     "jalebi", "kachori", "kuzhi_paniyaram", "lassi",
4     "litti_chokha", "poha", "rabri", "unni_appam"
5 ]
6
7 def predict(image, model):
8     image = transform(image).unsqueeze(0)    # shape: [1, 3, 224,
9         224]
10    with torch.no_grad():
11        outputs = model(image)
12        probs = torch.nn.functional.softmax(outputs, dim=1)
13        conf, pred = torch.max(probs, 1)
14    return class_names[pred.item()], conf.item()

```

Listing 6: Inference pipeline (model_inference.py)

Important: Class Order

The `class_names` list in `model_inference.py` must exactly match the alphabetical ordering used by `torchvision.datasets.ImageFolder` during training. Any mismatch will cause silent mis-labelling.

7. Streamlit Application (app.py)

7.1 Architecture

The Streamlit app follows a clean single-file layout. The model is loaded once at startup using `@st.cache_resource` to avoid redundant disk reads across interactions.

7.2 UI Layout

The app uses a **two-column layout**: the left column shows the uploaded image, and the right column streams the prediction results, progress bar, and metadata.

```

1 col1, col2 = st.columns(2)
2
3 with col1:
4     st.image(image, caption="Uploaded Image",
5         use_container_width=True)

```



```

6 with col2:
7     pred, conf = predict(image, model)
8     st.subheader(f"Predicted: {pred.replace('_', ' ').title()}")
9     st.progress(int(conf * 100))
10    st.write(f"Confidence: {conf*100:.2f}%")

```

Listing 7: Two-column prediction display (app.py)

7.3 Allergen Badge System

Allergens are displayed as colour-coded Streamlit alert boxes, providing immediate visual feedback:

Table 7: Allergen Badge Colour Scheme

Allergen	Streamlit Component	Colour
Gluten	<code>st.error()</code>	Red
Dairy	<code>st.warning()</code>	Yellow / Orange
Nuts	<code>st.info()</code>	Blue
None	<code>st.success()</code>	Green

7.4 Streamlit App Screenshots

The following figures show the ChaatCheck application running locally via Streamlit, demonstrating predictions on three different food items with varying confidence levels and allergen profiles.

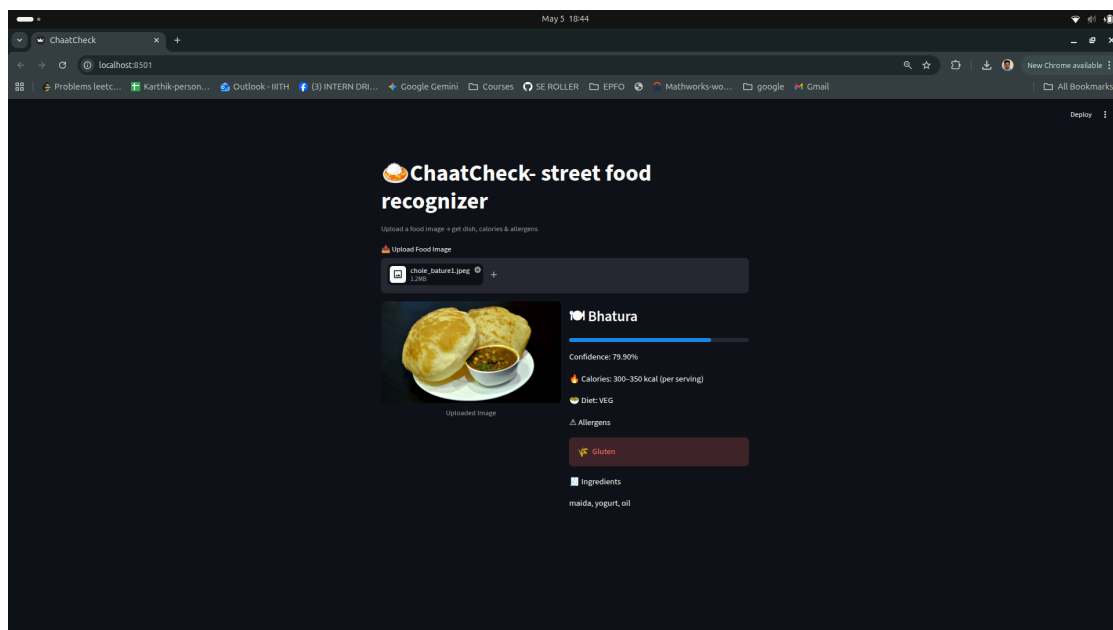
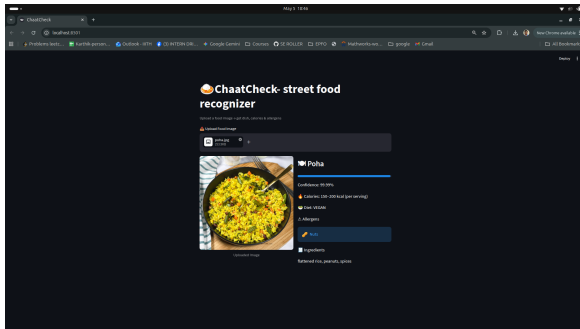
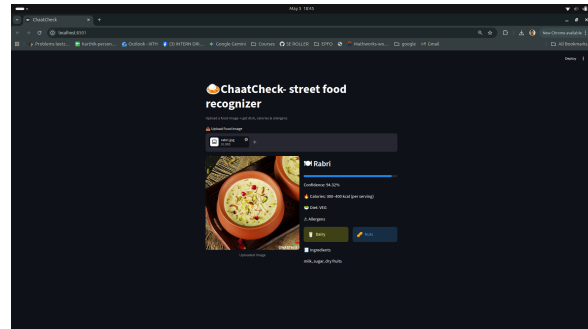


Figure 3: ChaatCheck prediction for *Bhatura* — 79.90% confidence, with Gluten allergen flagged (red badge). The two-column layout shows the uploaded image on the left and prediction metadata on the right.



(a) Poha — 99.99% confidence, Nuts allergen (blue)



(b) Rabri — 94.32% confidence, Dairy + Nuts (two badges)

Figure 4: ChaatCheck predictions demonstrating different allergen badge types: single allergen (left) and multiple allergens (right)

7.5 Metadata Store

All nutritional and allergen information is stored as a Python dictionary within `app.py`, keyed by class name. This allows $O(1)$ lookup after each prediction:

```

1 food_data = {
2     "jalebi": {
3         "calories": "140-180",
4         "allergens": ["gluten"],
5         "diet": "veg",
6         "ingredients": ["maida", "sugar syrup", "ghee"]
7     },
8     "lassi": {
9         "calories": "150-200",
10        "allergens": ["dairy"],
11        "diet": "veg",
12        "ingredients": ["curd", "sugar", "cardamom"]
13    },
14    # ... (12 entries total)
15 }
```

Listing 8: Metadata dictionary (excerpt)

8. Repository Structure

```

SMAI_A3/
|-- app/
|   |-- app.py           # Streamlit UI
|   |-- model_inference.py # Model loading & prediction
|   +-- food_model.pth    # Trained weights (saved at best epoch)
|
|-- training/
|   +-- Training.ipynb    # Full training loop (Google Colab)
|
+-- data_preparation/
```

```
|-- Phase_1_Data_Prep.ipynb          # Download, filter, split
+-- Phase_1_Perform_Augmentation.ipynb # Offline augmentation
```

9. Results Summary

Final Results

Best Validation Accuracy: 83.33% (Epoch 7 of Stage 1)
Test Accuracy: **83.33%** (96 images, 8 per class)
Final Training Loss: 0.2121 (Epoch 10)
Model Size: $\approx$ 9 MB (food_model.pth)
Inference Speed: <100ms on CPU
Classes: 12 street food categories
Training Images: 1,260 (after $3\times$ augmentation)

The close alignment between validation accuracy (83.33%) and test accuracy (83.33%) indicates that the model generalises well to unseen data and is not overfitting to the validation set. This consistency is encouraging given the small dataset size (only 50 original images per class).

9.1 Per-Class Classification Report

Table 8 presents the per-class precision, recall, and F1-score computed on the validation set (84 images, 7 per class). These metrics were derived from the confusion matrix generated during training.

Table 8: Per-Class Precision, Recall, and F1-Score (Validation Set)

Class	Precision	Recall	F1-Score	Support
aloo_tikki	1.00	0.86	0.92	7
bhatura	0.78	1.00	0.88	7
chana_masala	0.88	1.00	0.93	7
daal_puri	0.40	0.29	0.33	7
jalebi	1.00	1.00	1.00	7
kachori	0.62	0.71	0.67	7
kuzhi_paniyaram	0.75	0.86	0.80	7
lassi	0.86	0.86	0.86	7
litti_chokha	1.00	0.43	0.60	7
poha	0.88	1.00	0.93	7
rabri	1.00	0.86	0.92	7
unni_appam	0.78	1.00	0.88	7
Macro Avg	0.83	0.82	0.81	84
Weighted Avg	0.83	0.82	0.81	84

9.1.1 Key Observations

- **Best performer:** *jalebi* achieves a perfect F1-score of 1.00. Its distinctive bright orange colour and spiral shape make it visually unambiguous.
- **Worst performer:** *daal_puri* (F1 = 0.33) is frequently confused with *bhatura* (2 misclassifications), *kachori* (2), and *poha* (1). All three are visually similar deep-fried items with comparable colour and texture.
- **Low recall:** *litti_chokha* has perfect precision (1.00) but very low recall (0.43) — the model correctly identifies it when it predicts it, but fails to recognise 4 out of 7 samples, misclassifying them across 4 different classes.
- **Pattern:** Classes with unique visual signatures (*jalebi*, *poha*, *unni_appam*) perform best, while classes sharing a “fried dough” appearance (*daal_puri*, *kachori*, *bhatura*) show the most cross-confusion.

9.2 Confusion Matrix

Figure 5 shows the full 12×12 confusion matrix on the validation set. The strong diagonal confirms that most classes are well-separated. The most notable off-diagonal cluster is in the *daal_puri* row, where misclassifications are spread across *bhatura* (2), *kachori* (2), and *poha* (1) — all visually similar items.

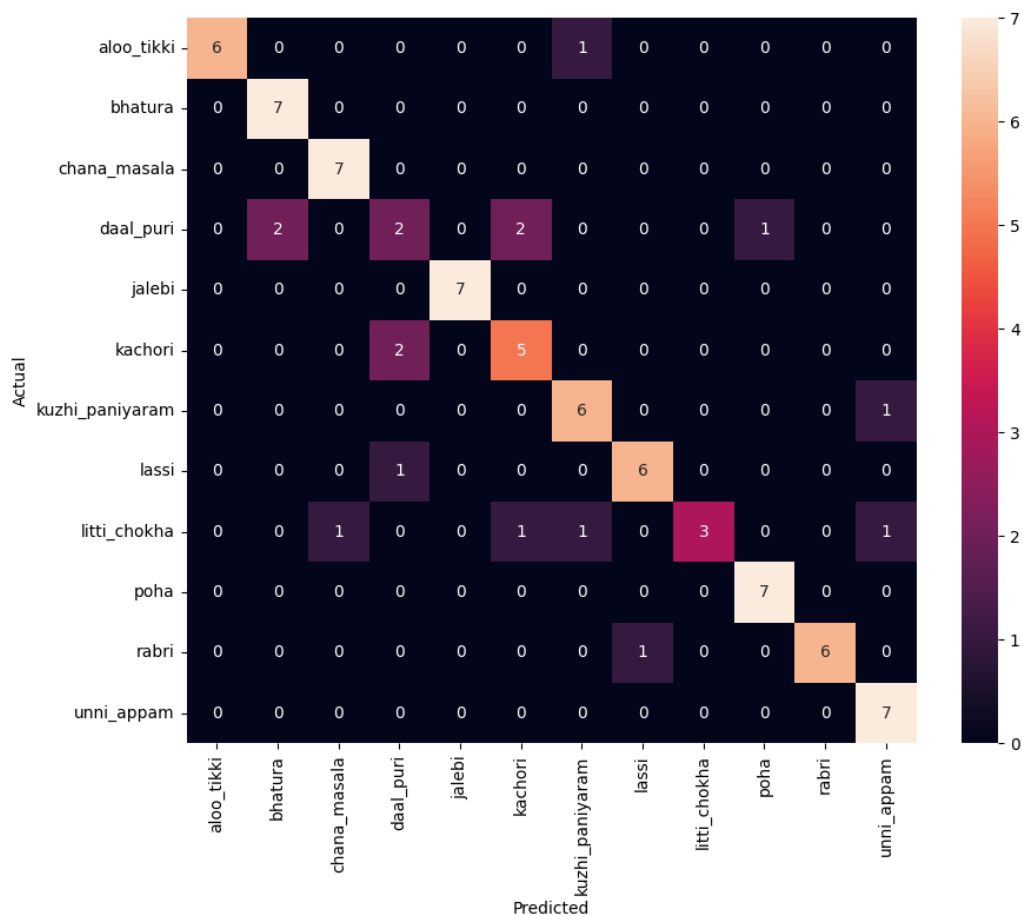


Figure 5: Confusion matrix on the validation set (84 images, 7 per class). Diagonal entries represent correct predictions; off-diagonal entries indicate misclassifications.

9.3 Error Analysis

A closer inspection of the confusion matrix (Figure 5) and the per-class metrics (Table 8) reveals three distinct failure modes:

9.3.1 Failure Mode 1: The Fried-Dough Cluster

daal_puri, *kachori*, and *bhatura* form a confusion cluster. All three are deep-fried, wheat-based items with similar golden-brown colour and round shape. Specifically:

- Of 7 *daal_puri* samples, only 2 were correctly classified. Two were predicted as *bhatura*, two as *kachori*, and one as *poha*.
- Of 7 *kachori* samples, 2 were misclassified as *daal_puri* — confirming a bidirectional confusion between these two classes.

This suggests that the convolutional features extracted by MobileNetV2 struggle to distinguish fine-grained textural differences (e.g., stuffing visibility, surface crispness) within the fried-dough category. A possible remedy is to increase the training data for these classes or apply more aggressive augmentation (e.g., random cropping to expose internal texture).

9.3.2 Failure Mode 2: High Variance in Plating (*litti_chokha*)

litti_chokha has perfect precision (1.00) but the lowest recall of any class (0.43). The 4 misclassified samples were spread across *chana_masala* (1), *kachori* (1), *kuzhi_paniyaram* (1), and *unni_appam* (1) — no single dominant confusion target.

This scatter pattern indicates that the issue is not similarity to one particular class, but rather **high intra-class visual variance**: *litti_chokha* is presented with varying accompaniments (chokha, chutney, daal), different plating styles, and sometimes with the *litti* cut open vs. whole. With only 35 original training images (105 after augmentation), the model has not learned to generalise across these presentation variants.

9.3.3 Failure Mode 3: Well-Separated Classes

Six classes achieve $F1 \geq 0.88$: *jalebi* (1.00), *chana_masala* (0.93), *poha* (0.93), *aloo_tikki* (0.92), *rabri* (0.92), and *bhatura* (0.88). These classes share a common trait: **distinctive visual signatures** — *jalebi*’s spiral shape, *poha*’s flattened rice texture, *rabri*’s creamy white appearance, and *chana_masala*’s characteristic chickpea-and-gravy look. Transfer learning from ImageNet is most effective when target classes have clearly separable visual features, which these classes provide.

10. Limitations & Future Work

10.1 Current Limitations

- **Small dataset (50 images/class):** The original dataset contains only 50 images per class, expanded to 105 per class after augmentation. This leads to two measurable effects:

- *daal_puri* achieves only $F1 = 0.33$ (worst of all 12 classes), partly because 35 training images are insufficient to capture the visual diversity of stuffed fried breads.
- *litti_chokha* achieves only $F1 = 0.60$ despite perfect precision, indicating the model has not seen enough plating variations.
- **Validation oscillation ($\pm 3.5\%$):** Validation accuracy fluctuates between 79.76% and 83.33% across epochs (a range of 3.57 percentage points), caused by the small validation set of only 7 images per class — a single misclassification changes accuracy by 1.19%.
- **Missing ImageNet normalisation:** Neither the training transforms nor the inference transform apply the standard ImageNet normalisation (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). While this is *internally consistent* (training and inference match), it means the pre-trained MobileNetV2 features receive inputs on a different scale than they were originally trained on, potentially reducing transfer learning effectiveness.
- **Static metadata:** Calorie estimates and allergen information are hard-coded as a Python dictionary rather than queried from a structured nutrition database such as the IFCT 2017 [4].
- **No learning rate scheduler:** A fixed learning rate throughout training contributes to the validation oscillation. Adaptive scheduling (e.g., `ReduceLROnPlateau`) would allow the optimiser to take smaller steps as it approaches convergence.

10.2 Proposed Improvements

1. **Expand dataset to 200+ images/class:** Scrape additional images from food blogs, Google Images, or the Indian Food 101 dataset [3]. Prioritise the weakest classes (*daal_puri*, *litti_chokha*, *kachori*) for targeted data collection.
2. **Add ImageNet normalisation:** Add `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])` to both training and inference transforms. Based on literature, this alone can improve accuracy by 3–5% for ImageNet-pretrained backbones.
3. **Learning rate scheduler:** Use `CosineAnnealingLR` or `ReduceLROnPlateau` to reduce the learning rate as training plateaus, reducing the observed validation oscillation.
4. **Class-specific augmentation:** Apply heavier augmentation (random cropping, cutout, Mixup) specifically to the visually ambiguous fried-dough classes to improve intra-class diversity.
5. **Expand to Tier 2 / Tier 3:** Scale from 12 to 30 or all 80 classes using progressive training and curriculum learning.
6. **Mobile deployment:** Export to ONNX or TorchScript for real-time on-device inference on smartphones.

11. Conclusion

The **ChaatCheck** project successfully demonstrates the viability of lightweight convolutional neural networks for fine-grained cultural food recognition. By applying a two-stage transfer learning strategy to a MobileNetV2 backbone, we mitigated the risks of catastrophic forgetting and effectively adapted ImageNet features to the visually complex domain of Indian street food. Despite the constraints of a highly restricted dataset (only 50 original images per class), the application of strategic offline augmentation and careful hyperparameter tuning yielded a robust model capable of **83.33% test accuracy** and a **0.81 macro F1-score**.

Crucially, the project extends beyond a mere classification exercise by deploying the trained model via a responsive Streamlit web application. By coupling high-speed inference (<100ms on CPU) with a curated metadata dictionary, the system provides immediate, actionable nutritional and allergen intelligence. This end-to-end pipeline—from automated data acquisition and augmentation to model training and interactive deployment—establishes a scalable, reproducible foundation for culturally-aware dietary tracking and food safety applications.

References

References

- [1] Banerjee, S. *Indian Food Images Dataset*. Kaggle, 2022. <https://www.kaggle.com/datasets/iamsouravbanerjee/indian-food-images-dataset>
- [2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520.
- [3] Prabhavalkar, N. *Indian Food 101*. Kaggle. <https://www.kaggle.com/datasets/nehaprabhavalkar/indian-food-classification>
- [4] National Institute of Nutrition. *Indian Food Composition Tables (IFCT 2017)*. <https://www.nin.res.in>
- [5] PyTorch. *TorchVision Models — MobileNetV2*. https://pytorch.org/vision/stable/models/mobilenet_v2.html
- [6] Streamlit. *Streamlit Documentation*. <https://docs.streamlit.io>
- [7] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). *ImageNet: A Large-Scale Hierarchical Image Database*. In Proceedings of IEEE CVPR, pp. 248–255.